

```
CREATE DATABASE shopping_db;  
USE shopping_db;
```

```
CREATE TABLE customers (  
customer_id INT PRIMARY KEY,  
customer_name VARCHAR(100),  
city VARCHAR(50),  
email VARCHAR(100)  
);
```

```
CREATE TABLE products (  
product_id INT PRIMARY KEY,  
product_name VARCHAR(100),  
category VARCHAR(50),  
price DECIMAL(10,2)  
);
```

```
CREATE TABLE orders (  
order_id INT PRIMARY KEY,  
customer_id INT,  
order_date DATE,  
payment_method VARCHAR(30),  
FOREIGN KEY (customer_id)  
REFERENCES customers(customer_id)  
);
```

```
INSERT INTO customers VALUES  
(1,'Rahul','Pune','rahul@gmail.com'),  
(2,'Sneha','Mumbai','sneha@gmail.com'),  
(3,'Amit','Nashik','amit@gmail.com'),  
(4,'Priya','Pune','priya@gmail.com');
```

```
INSERT INTO products VALUES  
(101,'Laptop','Electronics',60000),  
(102,'Mouse','Electronics',1000),  
(103,'Keyboard','Electronics',2500),  
(104,'Chair','Furniture',8000),  
(105,'Desk','Furniture',12000);
```

```
SELECT *  
FROM customers;
```

```
SELECT customer_name, city  
FROM customers;
```

```
SELECT *  
FROM customers  
WHERE city = 'Pune';
```

```
SELECT *  
FROM products  
WHERE price > 10000;
```

```
SELECT *  
FROM products  
ORDER BY price DESC;  
ORDER BY price ASC;
```

```
SELECT *  
FROM products  
ORDER BY price DESC  
LIMIT 3;
```

```
SELECT COUNT(*)  
FROM customers;
```

```
SELECT AVG(price)  
FROM products;
```

```
COUNT()  
SUM()  
AVG()  
MIN()  
MAX()
```

```
SELECT category, AVG(price)  
FROM products  
GROUP BY category;
```

```
SELECT category, MAX(price)  
FROM products  
GROUP BY category;
```

```
SELECT category, AVG(price)  
FROM products  
GROUP BY category;
```

```
SELECT category, AVG(price)
FROM products
GROUP BY category
HAVING AVG(price) > 10000;

WHERE price > 10000

HAVING AVG(price) > 10000
```

Level 1

1. Find all customers from Mumbai.
2. Find products costing more than ₹5,000.
3. Display products from highest to lowest price.

Level 2

4. Find average product price by category.
5. Find the number of products in each category.
6. Show categories having more than 2 products.

Level 3

7. Display customer names with their orders.
8. Display all customers, including those without orders.
9. Find products costing more than the average product price.

Level 4

10. Classify products as Affordable , Medium , and Expensive .
11. Rank products by price within each category.
12. Create a view showing customer name and order details.

```
SELECT
c.customer_name,
o.order_id,
o.order_date
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

```
SELECT
c.customer_name,
```

```
o.order_id  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id;
```

```
SELECT  
c.customer_name,  
o.order_id  
FROM customers c  
RIGHT JOIN orders o  
ON c.customer_id = o.customer_id;
```